

WireGuard VPN Server with WGDashboard & Experimental Device Validation

Self-hosted WireGuard VPN server setup using WGDashboard for peer management, along with an experimental machine-bound access validation system built using Python, Flask, Gunicorn, and Linux networking tools.

Overview

This project demonstrates:

- WireGuard VPN server setup on Ubuntu
- WGDashboard installation and peer management
- Full-tunnel VPN routing
- Mobile and laptop VPN connectivity
- Experimental device-bound access validation
- Machine-ID based device verification
- Production-style Flask API deployment using Gunicorn + systemd

The main goal of this project was to:

1. Build a self-hosted VPN server using WireGuard
 2. Explore ways to restrict VPN configuration usage to a single approved device
 3. Understand real-world VPN architecture and access-control concepts
-

Features

WireGuard VPN Server

- Self-hosted VPN server
- Secure encrypted tunnels
- Mobile and desktop support
- Full internet routing through VPN server
- Easy peer management using WGDashboard
- Ubuntu server deployment

WGDashboard Integration

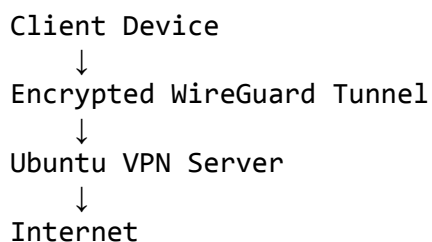
- Web-based VPN management UI
- Peer creation and deletion
- QR code generation for mobile devices
- VPN statistics and peer monitoring
- Simple client configuration management

Experimental Device Validation System

- Machine-ID based first-device registration
 - Centralized validation API
 - Custom VPN launcher script
 - Device mismatch detection
 - Production deployment using Gunicorn + systemd
-

Project Architecture

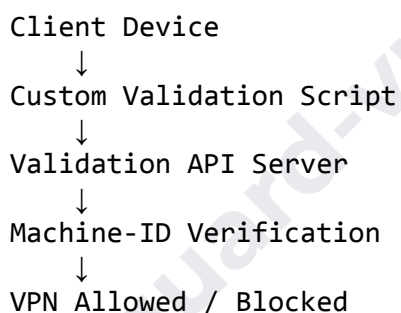
WireGuard VPN Flow



When VPN is connected:

- All internet traffic routes through the Ubuntu VPN server
 - Public IP changes to the server IP
 - Client traffic stays encrypted between device and server
-

Experimental Device Validation Flow



Workflow:

1. User runs a custom VPN launcher script
2. Script reads the Linux machine-id
3. Machine-id is sent to validation API
4. Server checks whether device is already registered
5. If device matches registered machine-id:
 - VPN starts

6. If machine-id does not match:
- VPN access is blocked
-

Technologies Used

- WireGuard
 - WGDashboard
 - Python
 - Flask
 - Gunicorn
 - systemd
 - Ubuntu Linux
 - Bash Scripting
 - Linux Networking
 - iptables
 - UFW Firewall
-

Project Structure

```
.
├── README.md
├── company-vpn-connect.sh
├── server.py
├── company-vpn.service
├── example-client.conf
├── example-wg0.conf
└── docs/
```

WireGuard Server Setup

1. Install WireGuard

```
sudo apt update
sudo apt install wireguard -y
```

2. Generate Server Keys

```
wg genkey | tee privatekey | wg pubkey > publickey
```

3. Create WireGuard Configuration

Example:

[Interface]

Address = 10.8.0.1/24

ListenPort = 51820

PrivateKey = SERVER_PRIVATE_KEY

PostUp = iptables -A FORWARD -i wg0 -j ACCEPT; iptables -A FORWARD -o wg0 -j ACCEPT; iptables -t nat -A POSTROUTING -o eno1 -j MASQUERADE

PostDown = iptables -D FORWARD -i wg0 -j ACCEPT; iptables -D FORWARD -o wg0 -j ACCEPT; iptables -t nat -D POSTROUTING -o eno1 -j MASQUERADE

Save as:

/etc/wireguard/wg0.conf

4. Enable IP Forwarding

sudo sysctl -w net.ipv4.ip_forward=1

Persist after reboot:

echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p

5. Start WireGuard

sudo systemctl enable wg-quick@wg0

sudo systemctl start wg-quick@wg0

Check status:

sudo wg

WGDashboard Setup

Official Project:

<https://github.com/donaldzou/WGDashboard>

WGDashboard was used for:

- Peer management
- Client config generation
- QR code generation
- VPN monitoring
- Easier administration

Client VPN Setup

Ubuntu/Linux Client

Place config inside:

`/etc/wireguard/`

Start VPN:

```
sudo wg-quick up test4_wg0
```

Disconnect VPN:

```
sudo wg-quick down test4_wg0
```

Mobile Client

1. Install WireGuard mobile app
 2. Import generated config or QR code
 3. Enable VPN
 4. Verify internet routes through VPN server
-

Experimental Device Validation System

Purpose

The experimental validation layer was created to explore ways of restricting a VPN configuration to only one approved device.

The idea was:

One VPN config → One Machine

Validation System Components

1. Validation API Server

Built using:

- Python
- Flask
- Gunicorn
- systemd

Responsibilities:

- Store machine registrations
 - Validate machine-id
 - Allow or block VPN startup
-

2. Custom VPN Launcher Script

Instead of directly using:

```
wg-quick up test4_wg0
```

VPN was started using:

```
~/company-vpn-connect.sh test4_wg0
```

Responsibilities:

- Read Linux machine-id
 - Contact validation server
 - Receive ALLOW / DENY response
 - Start VPN only if approved
-

3. Machine-ID Registry

Server stores:

```
{  
  "test4_wg0": "machine-id-value"  
}
```

This binds a VPN profile to the first machine that uses it.

Validation Workflow

First Device Registration

Laptop



Runs validation script



Machine-ID sent to server



No existing registration found



Machine-ID stored



VPN Allowed

Another Device Using Same Config

Another Laptop

↓
Runs same config
↓
Different Machine-ID detected
↓
Validation Failed
↓
VPN Blocked

Validation API Deployment

Run with Gunicorn

```
gunicorn -w 2 -b 0.0.0.0:5005 server:app
```

systemd Service

Example service file:

```
[Unit]
Description=Company VPN Validation API
After=network.target

[Service]
User=root
WorkingDirectory=/opt/company-vpn
ExecStart=/usr/bin/gunicorn -w 2 -b 0.0.0.0:5005 server:app
Restart=always

[Install]
WantedBy=multi-user.target
```

Enable Service

```
sudo systemctl daemon-reload
sudo systemctl enable company-vpn
sudo systemctl start company-vpn
```

Check status:

```
sudo systemctl status company-vpn
```

Testing Performed

VPN Testing

- Mobile VPN connection
 - Ubuntu laptop connection
 - Public IP verification
 - Internet routing through VPN server
 - Full-tunnel routing
-

Experimental Validation Testing

- First-device registration
 - Machine-ID verification
 - Device mismatch detection
 - Registry updates
 - Validation API testing
 - Unicorn deployment testing
 - systemd auto-start testing
-

Important Limitation

The experimental machine-binding system works only through the custom launcher script.

Direct imports into standard WireGuard clients/apps can bypass the validation layer because WireGuard itself does not natively support device identity enforcement.

This project was mainly built as:

Proof of Concept / Experimental Access-Control Layer

to better understand VPN security and device validation concepts.

Future Improvements

Potential future improvements:

- Headscale + Tailscale integration
- Device approval workflow
- Zero Trust architecture
- TPM-backed device identity
- MFA / SSO integration
- Centralized device management

- Server-side access enforcement
 - Enterprise-style VPN architecture
-

Headscale + Tailscale Research

During this project, modern VPN access architectures such as:

- Headscale
- Tailscale
- Firezone
- NetBird

were also explored.

These platforms provide:

- Device-based access control
- Centralized approval systems
- Revocation support
- Modern Zero Trust style networking

instead of relying only on shareable VPN config files.

Demo Video Covers

The demo video demonstrates:

- WireGuard server setup
 - WGDashboard overview
 - Peer creation
 - Mobile VPN connection
 - Ubuntu laptop VPN connection
 - Public IP verification
 - Internet routing through VPN server
 - Experimental machine-bound validation workflow
 - Device mismatch blocking
 - Validation architecture explanation
 - Limitations and future improvements
-

Security Note

Do NOT upload:

- Real private keys
- Real VPN configs
- Production IP addresses
- Sensitive credentials
- Real peer information
- Production registry files

Only upload sanitized example configurations.

Learning Objectives

This project helped explore:

- VPN architecture
 - Linux networking
 - WireGuard setup
 - Secure tunneling
 - NAT and IP forwarding
 - Firewall configuration
 - Flask API deployment
 - Gunicorn + systemd
 - Device validation concepts
 - Experimental access-control systems
 - Zero Trust networking ideas
-

Repository Topics

wireguard
vpn
wgdashboard
linux
ubuntu
networking
cybersecurity
devops
homelab
self-hosted
python
flask
gunicorn
systemd
security

Repository Description

Self-hosted WireGuard VPN server with WGDashboard and an experimental machine-bound access validation system.

License

MIT License

wireguard-vpn-server | Tirth R. Modi